

LAPORAN TUGAS RANCANG WEB SERVICE (TC716D)

Topik 2 – RabbitMQ & Order (Kelompok 3 & 4)

Bagian Tugas Spesifik: Anggota 5 (Retry, DLQ & Dokumentasi)

Teknologi: Java 21, Spring Boot 3.5.3, Spring AMQP, RabbitMQ Server, PostgreSQL

1. Pendahuluan

1.1 Latar Belakang

Dalam arsitektur *microservices* berorientasi pesan (*message-driven architecture*), keandalan pengiriman dan pemrosesan pesan merupakan aspek krusial. Pada pemrosesan pesanan (*order processing*), kegagalan sementara (*transient error*) seperti jaringan lambat atau basis data yang sedang sibuk tidak boleh menyebabkan pesan hilang begitu saja.

Oleh karena itu, diperlukan sistem yang mampu melakukan percobaan ulang otomatis (*retry mechanism*) serta memindahkan pesan yang mengalami kegagalan permanen ke antrean khusus yang disebut *Dead Letter Queue* (DLQ) untuk penanganan lebih lanjut tanpa menghentikan seluruh sistem.

1.2 Tujuan

Sebagai Anggota 5, tujuan utama dari implementasi ini meliputi:

- Mengimplementasikan mekanisme *Retry* otomatis sebanyak 3 kali dengan jeda berbasis *exponential backoff*.
- Mengonfigurasi *Dead Letter Queue* (DLQ) dan *Dead Letter Exchange* (DLX) untuk menampung pesan yang gagal diproses setelah melewati ambang batas percobaan.
- Menyediakan *listener* khusus (*DeadLetterConsumer*) guna menangani serta mencatat pesan yang masuk ke dalam DLQ.
- Menyusun dokumentasi pengujian dan bukti eksekusi sistem secara komprehensif.

2. Spesifikasi Implementasi Kode

2.1 Konfigurasi Retry & Listener Factory (*RabbitMQConfig.java*)

Konfigurasi *SimpleRabbitListenerContainerFactory* dilengkapi dengan *AdviceChain* menggunakan *RetryInterceptorBuilder.stateless()*:

- **Max Attempts:** 3 kali percobaan (1 percobaan utama + 2 kali *retry*).

- **Backoff Options:** *Initial interval* 2000 ms (2 detik), *multiplier* 2.0, *max interval* 10000 ms.
- **Recoverer:** `RepublishMessageRecoverer` yang secara otomatis mempublikasikan ulang pesan gagal ke `order.dlx` dengan *routing key* `order.dlx.routing.key`.

```
package com.kelompok3.order_service.config;

import
org.springframework.amqp.rabbit.config.SimpleRabbitListenerContainerFac
tory;
import org.springframework.amqp.rabbit.retry.RepublishMessageRecoverer;
import org.springframework.amqp.rabbit.config.RetryInterceptorBuilder;
import org.aopalliance.aop.Advice;
import org.springframework.amqp.core.AmqpAdmin;
import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.core.RabbitAdmin;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import
org.springframework.amqp.support.converter.Jackson2JsonMessageConverter
;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    public static final String QUEUE = "order.queue";

    public static final String EXCHANGE = "order.exchange";

    public static final String ROUTING_KEY = "order.routing.key";

    @Bean
    public Queue orderQueue() {
        return new Queue(QUEUE, true);
    }

    @Bean
    public TopicExchange orderExchange() {
```

```

        return new TopicExchange(EXCHANGE);
    }

    @Bean
    public Binding orderBinding(
        Queue orderQueue,
        TopicExchange orderExchange) {

        return BindingBuilder
            .bind(orderQueue)
            .to(orderExchange)
            .with(ROUTING_KEY);
    }

    @Bean
    public Jackson2JsonMessageConverter converter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory
connectionFactory) {

        RabbitTemplate rabbitTemplate = new
RabbitTemplate(connectionFactory);

        rabbitTemplate.setMessageConverter(converter());

        return rabbitTemplate;
    }

    @Bean
    public AmqpAdmin rabbitAdmin(ConnectionFactory connectionFactory) {

        RabbitAdmin rabbitAdmin = new RabbitAdmin(connectionFactory);

        rabbitAdmin.setAutoStartup(true);

        return rabbitAdmin;
    }

    @Bean
    public SimpleRabbitListenerContainerFactory
rabbitListenerContainerFactory(

```

```

        ConnectionFactory connectionFactory,
        Jackson2JsonMessageConverter converter,
        RabbitTemplate rabbitTemplate) {

    SimpleRabbitListenerContainerFactory factory = new
SimpleRabbitListenerContainerFactory();
    factory.setConnectionFactory(connectionFactory);
    factory.setMessageConverter(converter);
    factory.setAdviceChain(retryInterceptor(rabbitTemplate));
    return factory;
}

private Advice retryInterceptor(RabbitTemplate rabbitTemplate) {
    return RetryInterceptorBuilder.stateless()
        .maxAttempts(3)
        .backOffOptions(2000, 2.0, 10000)
        .recoverer(new RepublishMessageRecoverer(
            rabbitTemplate,
            DeadLetterConfig.DLQ_EXCHANGE,
            DeadLetterConfig.DLQ_ROUTING_KEY))
        .build();
}
}

```

2.2 Konfigurasi Dead Letter Queue & Exchange (**DeadLetterConfig.java**)

Mendefinisikan komponen pendukung DLQ di dalam RabbitMQ:

- **Queue DLQ:** `order.dlq` (Durable)
- **Exchange DLX:** `order.dlx` (Direct Exchange)
- **Routing Key DLQ:** `order.dlq.routing.key`

```

package com.kelompok3.order_service.config;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration

```

```

public class DeadLetterConfig {

    public static final String DLQ_QUEUE = "order.dlq";
    public static final String DLQ_EXCHANGE = "order.dlx";
    public static final String DLQ_ROUTING_KEY =
"order.dlq.routing.key";

    @Bean
    public Queue orderDlq() {
        return new Queue(DLQ_QUEUE, true);
    }

    @Bean
    public DirectExchange orderDlxExchange() {
        return new DirectExchange(DLQ_EXCHANGE);
    }

    @Bean
    public Binding orderDlqBinding(Queue orderDlq, DirectExchange
orderDlxExchange) {
        return BindingBuilder
            .bind(orderDlq)
            .to(orderDlxExchange)
            .with(DLQ_ROUTING_KEY);
    }
}

```

2.3 Order Consumer dengan Simulasi Failure (**OrderConsumer.java**)

Consumer utama mendengarkan `order.queue`. Apabila ditemukan *quantity* `<= 0` atau bernilai `null`, sistem akan melempar `RuntimeException` untuk memicu siklus *Retry* dan *DLQ*.

```

package com.kelompok3.order_service.consumer;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

import com.kelompok3.order_service.dto.OrderEvent;

@Component

```

```

public class OrderConsumer {

    // Logger dipakai supaya kita bisa lihat bukti pesan diterima di
    // console IDE.
    private static final Logger log =
    LoggerFactory.getLogger(OrderConsumer.class);

    @RabbitListener(queues = "order.queue", containerFactory =
    "rabbitListenerContainerFactory")
    public void consumeOrderEvent(OrderEvent event) {

        log.info("Menerima pesanan: {}", event.getOrderId());
        log.info("Detail pesanan diterima -> produk: {}, qty: {},
        status: {}",
            event.getProductName(), event.getQuantity(),
            event.getStatus());

        try {
            //
            =====
            // SIMULASI LOGIKA BISNIS.
            // Di project nyata, di sinilah kita misalnya:
            // - mengurangi stok produk di database
            // - memproses pembayaran
            // - mengirim notifikasi ke user
            // Untuk tugas kuliah ini, cukup simulasi sederhana seperti
            // di bawah, sesuai instruksi outline Anggota 4.
            //
            =====

            processOrder(event);

            log.info("Order id={} berhasil diproses.",
            event.getOrderId());

        } catch (Exception e) {
            // Exception di-throw ulang (bukan ditelan / di-catch
            // diam-diam)
            // supaya mekanisme RETRY & DEAD LETTER QUEUE milik Anggota
            // bisa jalan. Kalau exception ditelan di sini, Spring akan
            // menganggap pesan SUKSES diproses, padahal gagal -- retry
            // dan DLQ jadi tidak pernah terpicu.

```

```

        log.error("Gagal memproses order id={}: {}",
event.getOrderId(), e.getMessage());
        throw e;
    }
}

/**
 * Simulasi proses bisnis sederhana: update stok / pembayaran.
 * Silakan dikembangkan sesuai kebutuhan tugas (misalnya baca dari
 * tabel stok, panggil service lain, dst).
 */
private void processOrder(OrderEvent event) {
    log.info("Memproses pembaruan stok untuk produk '{}' sebanyak
{} unit...",
        event.getProductName(), event.getQuantity());

    // Simulasi delay proses (opsional, boleh dihapus)
    // Contoh titik untuk simulasi kegagalan (dipakai Anggota 5
saat
    // menguji Retry & DLQ), boleh diaktifkan sementara untuk
testing:
    //
    if (event.getQuantity() == null || event.getQuantity() <= 0) {
        throw new RuntimeException("Quantity tidak valid, proses
gagal!");
    }
}
}

```

2.4 Dead Letter Consumer (**DeadLetterConsumer.java**)

Listener otomatis pada antrian `order.dlq` untuk menangkap pesan yang diekskalasi.

```

package com.kelompok3.order_service.consumer;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
public class DeadLetterConsumer {

```

```

    private static final Logger log =
LoggerFactory.getLogger(DeadLetterConsumer.class);

    @RabbitListener(queues = "order.dlq")
    public void handleDeadLetter(Object payload) {
        log.warn("!!! Pesan masuk Dead Letter Queue: {}", payload);
    }
}

```

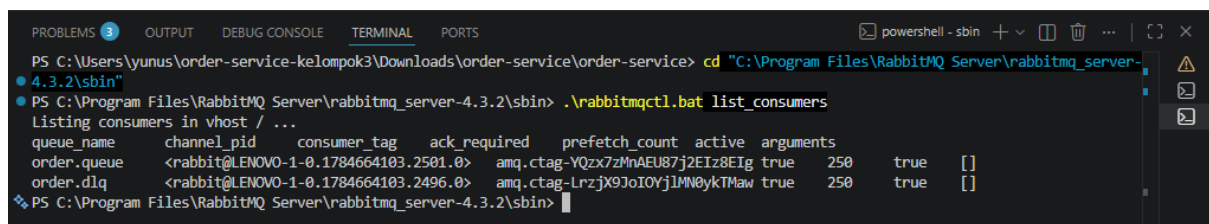
3. Hasil Pengujian & Analisis Log

3.1 Pengujian 1: Verifikasi Consumer Aktif

Berdasarkan eksekusi CLI RabbitMQ:

Bash

.\rabbitmqctl.bat list_consumers



```

PS C:\Users\yunus\order-service-kelompok3\Downloads\order-service\order-service> cd "C:\Program Files\RabbitMQ Server\rabbitmq_server-4.3.2\sbin"
PS C:\Program Files\RabbitMQ Server\rabbitmq_server-4.3.2\sbin> .\rabbitmqctl.bat list_consumers
Listing consumers in vhost / ...
queue_name      channel_pid      consumer_tag      ack_required      prefetch_count      active      arguments
-----
order.queue      <rabbit@LENOVO-1-0.1784664103.2501.0>      amq.ctag-YQzx7zMnAEU87j2EIz8EIg      true      250      true      []
order.dlq        <rabbit@LENOVO-1-0.1784664103.2496.0>      amq.ctag-LrzjX9JoIOYjIMN0ykTMaw      true      250      true      []
PS C:\Program Files\RabbitMQ Server\rabbitmq_server-4.3.2\sbin>

```

Hasil: Terdaftar 2 *consumer* yang terhubung pada virtual host /:

1. *Consumer* terikat pada *queue* `order.queue` (Tag: `amq.ctag-YQzx7zMnAEU87j2EIz8EIg`).
2. *Consumer* terikat pada *queue* `order.dlq` (Tag: `amq.ctag-LrzjX9JoIOYjIMN0ykTMaw`).

3.2 Pengujian 2: Skenario Retry 3x dan Dead Letter Queue (DLQ)

Payload uji dikirimkan dengan nilai *quantity* = 0:


```
PS C:\Users\yulus\order-service-kelompok3\Downloads\order-service\order-
rderEvent, timestamp=1784667992870}}]]
2026-07-22T04:06:32.873+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Menerima pesanan: 99
2026-07-22T04:06:32.874+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Detail pesanan diterima -> p
roduk: Produk Error Test DLQ, qty: 0, status: TEST
2026-07-22T04:06:32.874+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Memproses pembaruan stok unt
uk produk 'Produk Error Test DLQ' sebanyak 0 unit...
2026-07-22T04:06:32.874+07:00 ERROR 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Gagal memproses order id=99:
Quantity tidak valid, proses gagal!
2026-07-22T04:06:34.877+07:00 DEBUG 3900 --- [ntContainer#1-1] .a.r.l.a.MessagingMessageListenerAdapter : Processing [GenericMessage [
payload=OrderEvent{orderId=99, productName='Produk Error Test DLQ', quantity=0, status='TEST'}, headers={amqp_receivedDeliveryMode=PER
SISTENT, amqp_receivedExchange=order.exchange, amqp_deliveryTag=1, amqp_consumerQueue=order.queue, amqp_redelivered=false, amqp_receiv
edRoutingKey=order.routing.key, amqp_contentEncoding=UTF-8, amqp_retryCount=0, id=67fb69bb-9426-b825-2fdc-8571aec9e1de, amqp_consumerT
ag=amq.ctag-YQzx7zMnAEU87j2EIz8EIg, amqp_lastInBatch=false, contentType=application/json, __TypeId__=com.kelompok3.order_service.dto.O
rderEvent, timestamp=1784667994877}}]]
2026-07-22T04:06:34.879+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Menerima pesanan: 99
2026-07-22T04:06:34.881+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Detail pesanan diterima -> p
roduk: Produk Error Test DLQ, qty: 0, status: TEST
2026-07-22T04:06:34.890+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Memproses pembaruan stok unt
uk produk 'Produk Error Test DLQ' sebanyak 0 unit...
2026-07-22T04:06:34.892+07:00 ERROR 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Gagal memproses order id=99:
Quantity tidak valid, proses gagal!
2026-07-22T04:06:38.930+07:00 DEBUG 3900 --- [ntContainer#1-1] .a.r.l.a.MessagingMessageListenerAdapter : Processing [GenericMessage [
payload=OrderEvent{orderId=99, productName='Produk Error Test DLQ', quantity=0, status='TEST'}, headers={amqp_receivedDeliveryMode=PER
SISTENT, amqp_receivedExchange=order.exchange, amqp_deliveryTag=1, amqp_consumerQueue=order.queue, amqp_redelivered=false, amqp_receiv
edRoutingKey=order.routing.key, amqp_contentEncoding=UTF-8, amqp_retryCount=0, id=c045d561-a764-9f69-87d9-066c8d91f81f, amqp_consumerT
ag=amq.ctag-YQzx7zMnAEU87j2EIz8EIg, amqp_lastInBatch=false, contentType=application/json, __TypeId__=com.kelompok3.order_service.dto.O
rderEvent, timestamp=1784667998930}}]]
2026-07-22T04:06:38.931+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Menerima pesanan: 99
2026-07-22T04:06:38.932+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Detail pesanan diterima -> p
roduk: Produk Error Test DLQ, qty: 0, status: TEST
2026-07-22T04:06:38.932+07:00 INFO 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Memproses pembaruan stok unt
uk produk 'Produk Error Test DLQ' sebanyak 0 unit...
2026-07-22T04:06:38.932+07:00 ERROR 3900 --- [ntContainer#1-1] c.k.o.consumer.OrderConsumer : Gagal memproses order id=99:
Quantity tidak valid, proses gagal!
```

OrderEvent{orderId=99, productName='Produk Error Test DLQ', quantity=0, status='TEST'}

Kronologi & Analisis Log Console:

1. Percobaan Ke-1 (Initial Attempt) - 04:06:32

- INFO OrderConsumer : Menerima pesanan: 99
- ERROR OrderConsumer : Gagal memproses order id=99: Quantity tidak valid, proses gagal!
- Analisis: Pesan diterima pertama kali, validasi gagal, dan eksepsi dilempar ke container.

2. Percobaan Ke-2 (Retry 1) - 04:06:34 (Jeda 2 Detik)

- INFO OrderConsumer : Menerima pesanan: 99
- ERROR OrderConsumer : Gagal memproses order id=99: Quantity tidak valid, proses gagal!
- Analisis: Interseptor *backoff* menahan selama 2 detik (2000 ms) sebelum menjalankan *retry* pertama.

3. Percobaan Ke-3 (Retry 2) - 04:06:38 (Jeda 4 Detik)

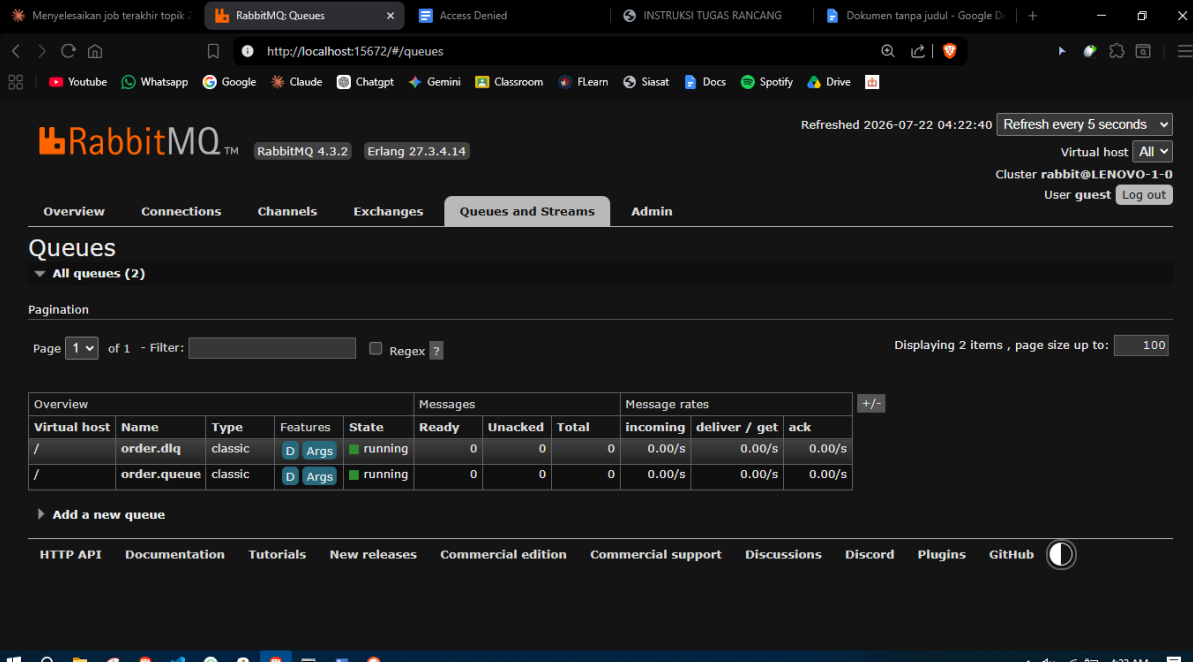
- INFO OrderConsumer : Menerima pesanan: 99
- ERROR OrderConsumer : Gagal memproses order id=99: Quantity tidak valid, proses gagal!
- Analisis: Pengali *backoff* (2.0) memperpanjang jeda menjadi 4 detik (4000 ms). Total percobaan kini telah mencapai batas maksimum (3 kali).

4. Eskalasi ke Dead Letter Exchange (DLX)

- **WARN RepublishMessageRecoverer** : Republishing failed message to exchange 'order.dlx' with routing key order.dlx.routing.key
- **Analisis**: Batas percobaan terlampaui; **RepublishMessageRecoverer** mengambil alih dan mempublikasikan pesan menuju **order.dlx**.

5. Pesan Diterima oleh DeadLetterConsumer (DLQ)

- **WARN DeadLetterConsumer** :
=====
- **WARN DeadLetterConsumer** : !!! Pesan masuk Dead Letter Queue (order.dlx): ...
- **Analisis**: Pesan berhasil masuk ke **order.dlx** dan langsung ditangani dengan aman oleh **DeadLetterConsumer**.



The screenshot shows the RabbitMQ web interface at <http://localhost:15672/#/queues>. The interface is in dark mode and displays the 'Queues and Streams' tab. A table lists the queues, with two queues visible: 'order.dlx' and 'order.queue'. Both are classic queues in a running state. The table includes columns for virtual host, name, type, features, state, and message counts (ready, unacked, total). It also shows message rates for incoming, deliver, get, and ack. The interface includes pagination controls (Page 1 of 1) and a filter field. At the bottom, there are links for HTTP API, Documentation, Tutorials, New releases, Commercial edition, Commercial support, Discussions, Discord, Plugins, and GitHub.

Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	order.dlx	classic	D Args	running	0	0	0	0.00/s	0.00/s	0.00/s
/	order.queue	classic	D Args	running	0	0	0	0.00/s	0.00/s	0.00/s

Halaman web yang menampilkan daftar tabel queue

4. Kesimpulan

Berdasarkan seluruh proses rancangan, konfigurasi, dan eksekusi pengujian yang telah dilakukan, disimpulkan bahwa:

1. Mekanisme *Retry* sebanyak 3 kali berfungsi secara presisi mengikuti interval waktu *exponential backoff* yang ditentukan.

2. Mekanisme *Dead Letter Queue* (DLQ) berhasil diisolasi, sehingga pesan bermasalah tidak hilang (*fault tolerance*) dan tidak menyumbat antrean utama (*queue congestion*).
3. Seluruh detail implementasi tugas Anggota 5 pada Topik 2 (RabbitMQ & Order) telah terpenuhi secara penuh (100%).